

AEDI System Architecture

Detailed Tech Stack Information

High Level Overview

Qwen model is used to interact with user and store data in PostgreSQL

Next.js is used in frontend UI and Django in backend for routing. User can either use dedicated api keys or the frontend UI to submit a detailed json payload

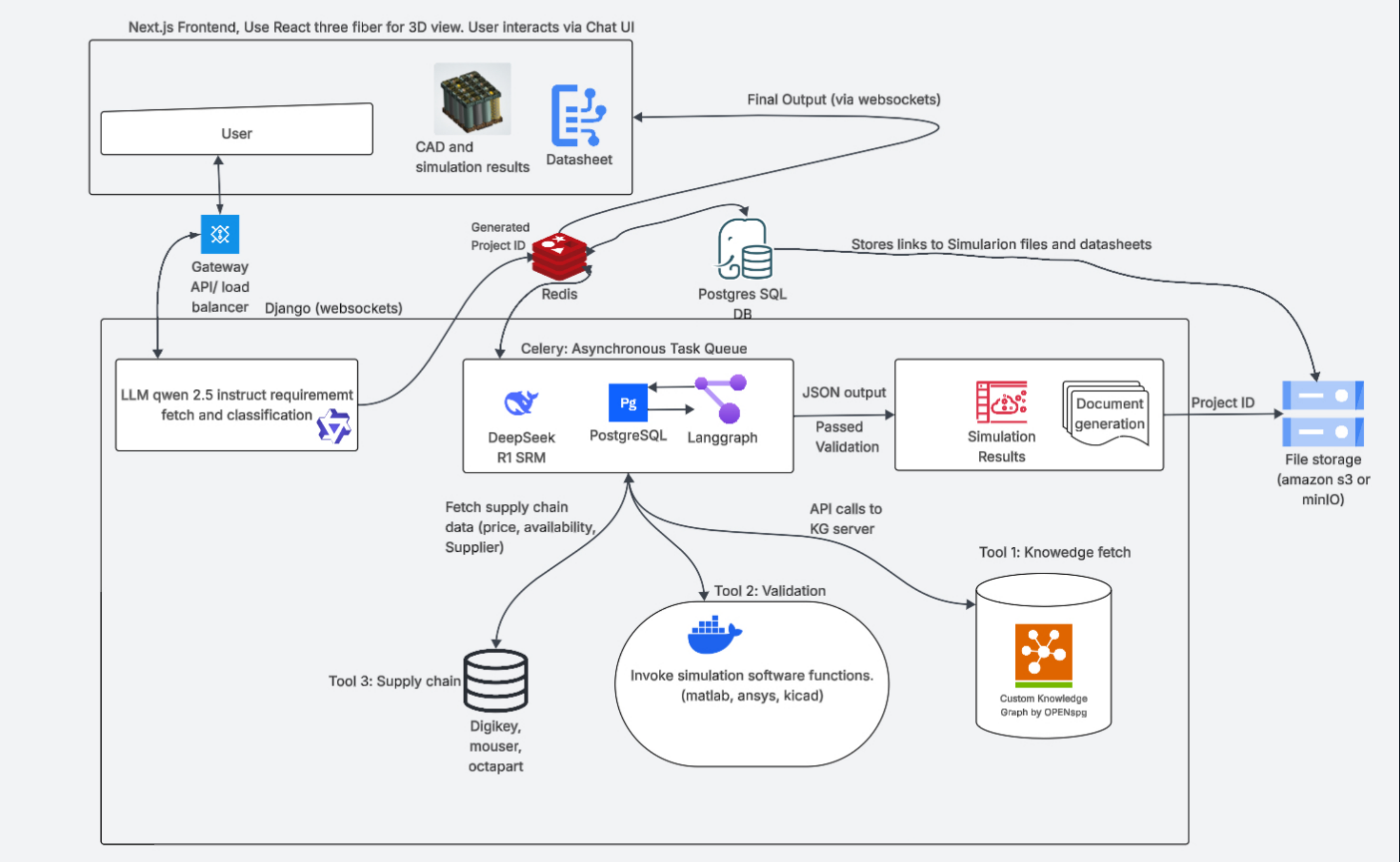
Celery (Asynchronous task Queue) is used to detach task from backend to run inferences and simulations

Our SRM along with knowledge graphs makes optimal design and chooses components. This is handled using langgraph state management

We will have different tools which are accessed by SRM via API.

Final results are stored in file storage systems and their links are saved in postgresSQL for easy download by the user.

Flow diagram

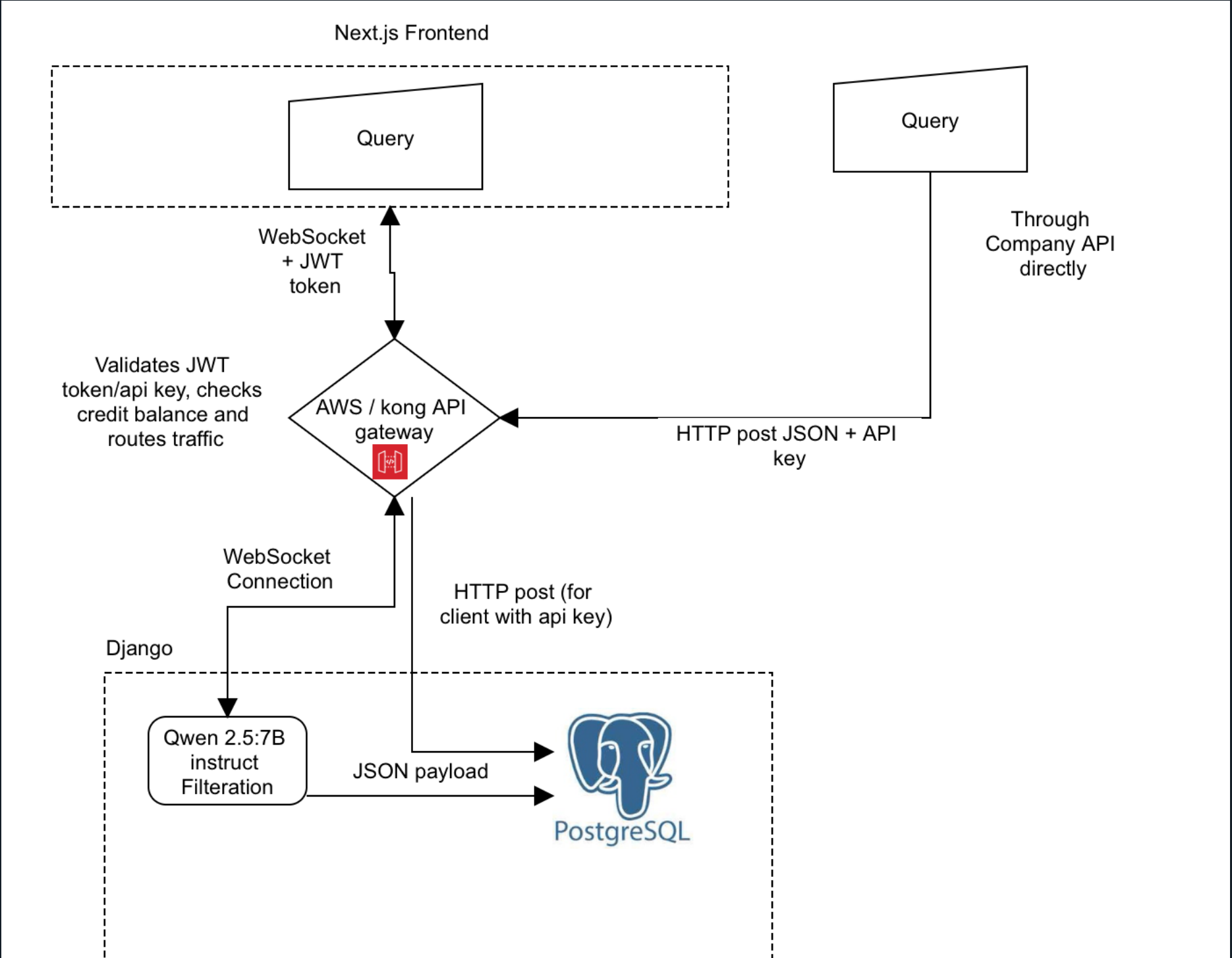


Input and Api Gateway

Client can give detailed json payload via API key or give details in natural language on frontend

Connections to backend passes via API gateway (NGNIX/KONG/AWS) to rate limiting, Authentication, Check credit balance before running query

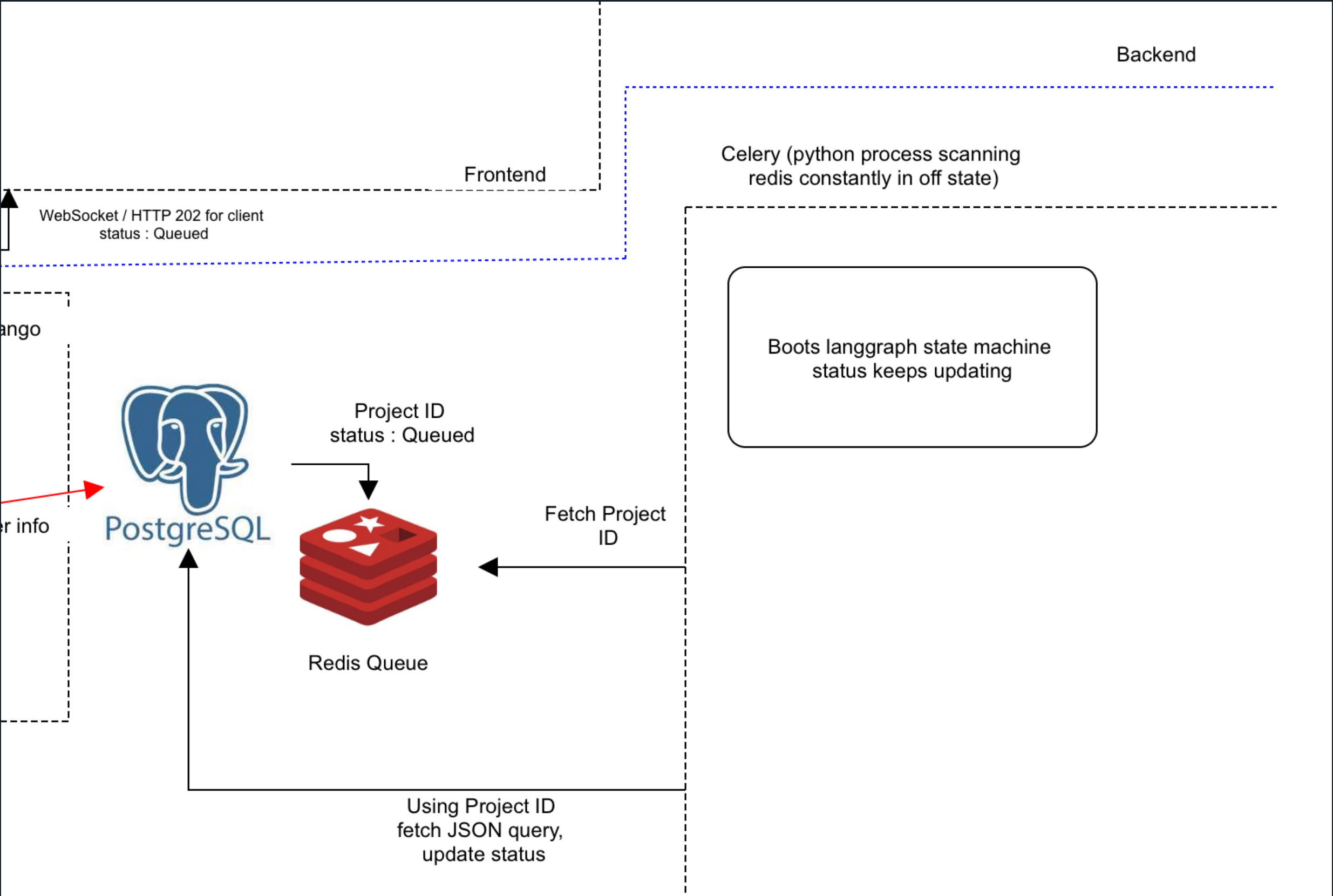
Task	Tech Stack
Frontend	Next.js + React3 fiber
Backend	Django, celery
API Gateway	NGNIX / KONG / AWS
Filteration	SFT Qwen2.5:7B instruct (via vllm or hugging face)
Storage	PostgresSQL, File storage



Data Store and Asynchronous task queue

JSON data is stored in PostgreSQL and project ID is pushed in redis. Celery framework reads it and initiate the langgraph state machine containing SRMs.

Task	Tech Stack
Web server layer	Django, Django Ninja, Uvicorn
Storage and message broker	PostgreSQL, redis-py for queue (only store project id)
Asynchronous task queue layer	Celery. This runs as completely separate service



SRM and Langgraph

Tool 1: KG by openspg

Tool 1 : Knowledge Builder

Tool 2 : Simulation and Validation

Tool 2: Simulation and Validation